

ReactJS

Zustand:

What is Zustand?

Zustand is a small, lightweight state management library for React. Its purpose is to let multiple components share state without prop drilling or excessive Context providers.

Feature	Zustand	Redux Toolkit
Boilerplate	Very little	Moderate
Learning curve	Easy	Medium
Store setup	Simple	More setup
Reducers	Optional	Required via slices
Actions	Usually not needed	Required
DevTools	Yes	Excellent
Middleware	Basic	Powerful
Async support	Simple	Excellent
Performance	Excellent	Excellent
Ecosystem	Smaller	Huge

How to install Zustand?

```
PS D:\ReactJS-revisited\learn-use-effect\React-useeffect-project> npm install zustand

added 1 package, and audited 232 packages in 4s

56 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
❖ PS D:\ReactJS-revisited\learn-use-effect\React-useeffect-project> |
```

Let's first create the store in zustand:

```
src > app > JS courseStore.js > useCourseStore
1  import create from 'zustand'
2  import { devtools, persist } from 'zustand/middleware'
3
4  const courseStore = (set) => ({
5    courses: [], // initial state
6    addCourse: (course) => {
7      set((state) => ({
8        courses: [course, ...state.courses],
9      })))
10   },
11
12   removeCourse: () => {
13     set((state) => ({
14       courses: state.courses.filter((c) => c.id !== courseId)
15     })))
16   },
17
18   toggleCourseStatus: (courseId) => {
19     set((state) => ({
20       courses: state.courses.map((course) => courseId === completed ? { ...course, completed: !course.completed } : course)
21     })))
22   }
23 })
24
25 const useCourseStore = create(
26   devtools(
27     persist(courseStore, {
28       name: "courses",
29     })
30   )
31 );
32 export default useCourseStore;
```

This code creates a **global store** using Zustand to manage a list of courses in a React app. It starts with an empty courses array and provides functions to add a course, remove a course, and toggle a course's completion status by updating the shared state with `set()`. The `persist` middleware saves the course data in the browser's local storage, so it remains available after a page refresh, while `devtools` lets you inspect and debug state changes using the Redux DevTools browser extension. Finally, the store is exported as `useCourseStore`, which any React component can use to access and update the shared course data.

Note: The `removeCourse` and `toggleCourseStatus` functions contain a few coding mistakes and would need small fixes to work correctly.

Now, we will create the form to take the input and store in the 'store'.

```
src > components > CourseForm.jsx > CourseForm
1  import React, { useState } from "react";
2  import useCourseStore from "../app/courseStore";
3  const CourseForm = () => {
4    const addCourse = useCourseStore((state) => state.addCourse);
5    const [courseTitle, setCourseTitle] = useState("");
6    console.log("CourseForm Rendered");
7    const handleCourseSubmit = () => {
8      if (!courseTitle) {
9        alert("Please add a course title");
10       return;
11     }
12     addCourse({
13       id: Math.ceil(Math.random() * 100000),
14       title: courseTitle,
15       completed: false,
16     });
17     setCourseTitle("");
18   };
19   return (
20     <div>
21       <h2>Add Course</h2>
22       <input type="text" placeholder="Enter course title" value={courseTitle} onChange={(e) => setCourseTitle(e.target.value)} />
23       <button onClick={handleCourseSubmit}>
24         Add Course
25       </button>
26     </div>
27   );
28 };
29
30 export default CourseForm;
```

Updated courseStore.js:

```
src > app > JS courseStore.js > courseStore > toggleCourseStatus > set() callback
1  import { create } from "zustand";
2  import { devtools, persist } from "zustand/middleware";
3  const courseStore = (set) => ({
4    // Initial State
5    courses: [],
6    // Add Course
7    addCourse: (course) =>
8      set((state) => ({
9        courses: [course, ...state.courses],
10      })),
11    // Remove Course
12    removeCourse: (courseId) =>
13      set((state) => ({
14        courses: state.courses.filter(
15          (course) => course.id !== courseId
16        ),
17      })),
18    toggleCourseStatus: (courseId) =>
19      set((state) => ({
20        courses: state.courses.map((course) =>
21          course.id === courseId
22            ? {
23              ...course,
24              completed: !course.completed,
25            }
26            : course
27        ),
28      })),
29  });
30  const useCourseStore = create(
31    devtools(
32      persist(courseStore, {
33        name: "courses",
34      })
35    )
36  );
37  export default useCourseStore;
```

Output:

The screenshot shows a web application interface on the left and the Chrome DevTools Application panel on the right.

Web Application Interface:

- My Course List**
- Add Course**: A form with an input field labeled "Enter course title" and a button labeled "Add Course".
- Courses**: A list showing "ReactJS Not Completed".

Chrome DevTools Application Panel:

- The **Storage** tab is selected, showing local storage for the extension.
- The **chrome-extension://http://localhost:5173** storage is expanded.
- The **courses** key is selected, showing its value in the right pane.

Key	Value
cart-storage	[{"state":{"products":[{"id":1,"name":"Ab","price":150,"summary":"Something A"}],"id":2...
courses	[{"state":{"courses":[{"id":"63870","title":"ReactJS","completed":false}],"version":0}
notes-app-theme	system
theme	light
zk-vault-tracked-notes	[{"noteId":"32c0bc52831d4df0bb8b570296fb5bc0","trackingToken":"1_NWjyd53a0X4_...